Web 2 – Séance 10

Modules et collections

Organisation des fichiers

- Définir des **types** dans `models/`
- Définir des **données** dans `data/`

```
src/
├── App.tsx        # Composant principal
├── models/        # Types : à quoi ressemble une recette ?
│   ├── recipe.ts
│   └── category.ts
├── data/          # Données : quelles sont les recettes ?
│   ├── recipes.ts
│   └── categories.ts
└── components/    # Affichage : comment montrer une recette ?
    ├── RecipeCard.tsx
    └── RecipeList.tsx
```

Le modèle Recipe

```ts
// models/recipe.ts
export interface Ingredient {
  name: string;
  quantity: number;
  unit: string;
}

export interface Recipe {
  id: number;
  title: string;
  description: string;
  imageUrl?: string;
  prepTime: number;   // minutes
  cookTime: number;   // minutes
  servings: number;
  difficulty: number; // 1 à 5
  categoryId: number;
  tags: string[];
  ingredients: Ingredient[];
  steps: string[];
  authorId: number;
}
```

Le modèle reprend **exactement** le `RecipeDTO` renvoyé par le backend MiamMiam.

Un module de données

```ts
// data/recipes.ts
import type { Recipe } from "../models/recipe";

export const recipes: Recipe[] = [
  {
    id: 1,
    title: "Pancakes moelleux",
    description: "Des pancakes épais et aérés pour un brunch réussi.",
    imageUrl: "https://images.unsplash.com/photo-1528207776546-365bb710ee93?w=800",
    prepTime: 10,
    cookTime: 15,
    servings: 4,
    difficulty: 1,
    categoryId: 3,
    tags: ["brunch", "sucré", "rapide"],
    ingredients: [
      { name: "farine", quantity: 250, unit: "g" },
      { name: "lait", quantity: 300, unit: "ml" },
    ],
    steps: ["Mélanger la farine, le sucre et la levure.", "Cuire dans une poêle chaude."],
    authorId: 2,
  },
  // ...
];
```

Passer un objet en prop

Plutôt que de passer chaque champ séparément, on peut passer la recette entière.

```
interface RecipeCardProps {
  recipe: Recipe;
}

const RecipeCard = ({ recipe }: RecipeCardProps) => {
  return (
    <Card>
      <CardMedia component="img" height="200" image={recipe.imageUrl} alt={recipe.title} />
      <CardContent>
        <Typography variant="h6" component="h2">{recipe.title}</Typography>
        <Typography variant="body2" color="text.secondary">
          {recipe.prepTime + recipe.cookTime} min
        </Typography>
      </CardContent>
    </Card>
  );
};
```

- Une seule prop, typée par le modèle
- Si le modèle évolue, l'interface des props suit automatiquement

Afficher un tableau de JSX

En JSX, une expression `{ ... }` peut contenir un **tableau de JSX** : React les affiche les uns après les autres.

```
const titles: JSX.Element[] = [
  <li key="1">Pancakes</li>,
  <li key="2">Carbonara</li>
];

return (
  <ul>{titles}</ul>
);
```

Équivalent à :

```
return (
  <ul>
    <li>Pancakes</li>
    <li>Carbonara</li>
  </ul>
);
```

Afficher un tableau de données avec map()

Pour afficher un tableau de données, on peut utiliser `map` pour le transformer en un tableau de JSX :

```
import { recipes } from "../data/recipes";

const RecipeTitles = () => {
  return (
    <ul>
      {recipes.map((recipe) => <li key={recipe.id}>{recipe.title}</li>)}
    </ul>
  );
};
```

- `recipes` est un tableau d'objets `Recipe`
- `map` appelle la fonction pour chaque recette et retourne un nouveau tableau
- Le résultat de `map` est un tableau d'objets `JSX.Element` que React peut afficher

Pourquoi une key ?

Quand les données **changent**, React ne recrée pas toute la page : il **compare** le nouveau JSX à l'ancien et ne modifie dans le DOM que ce qui a changé (**réconciliation**).

Dans une liste, React doit savoir quel élément correspond à quel élément :

```
Avant :  [Pancakes, Carbonara]
Après :  [Guacamole, Pancakes, Carbonara]
```

- Sans key, React compare par **position** : il croit que `Pancakes` est devenu `Guacamole`, `Carbonara` est devenu `Pancakes`, et qu'un élément `Carbonara` a été ajouté à la fin. Les composants enfants sont recréés et leur état est perdu.
- Avec un attribut `key` différent pour chaque élément, il sait que Pancakes et Carbonara n'ont pas changé et il insère correctement Guacamole au début.

Oubli de la key → warning dans la console : *Each child in a list should have a unique "key" prop*

> La key permet de **préserver l'état** des composants enfants quand la liste change

Quelle key choisir ?

- **Index** : acceptable seulement si la liste ne change jamais d'ordre ni de taille (rare) : ⚠️

```
{recipe.steps.map((step, index) => <li key={index}>{step}</li>)}
```

- **Nom ou titre** : acceptable seulement si on est sûr qu'il est unique et stable (rare) : ⚠️

```
{recipes.map((recipe) => <RecipeCard key={recipe.title} recipe={recipe} />)}
```

- **Identifiant unique** venant des données ✅

```
{recipes.map((recipe) => <RecipeCard key={recipe.id} recipe={recipe} />)}
```

Règles :

- Elle doit être **unique** : pas deux éléments avec la même key dans le même tableau
- Elle doit être **stable** : si l'élément reste dans la liste, sa key ne doit pas changer
  - L'indice change quand on insère ou supprime un élément, un titre peut être modifié
  - L'id est stable : il ne change jamais pour un élément donné

Key avec un composant

La key n'est pas transmise au composant : ce n'est pas une prop.

```
// RecipeList.tsx
<Stack spacing={2}>
  {recipes.map((recipe) => <RecipeCard key={recipe.id} recipe={recipe} />)}
</Stack>
```

```
// RecipeCard.tsx
interface RecipeCardProps {
  recipe: Recipe;
}
```

Key avec plusieurs composants imbriqués

La key doit être sur le composant **directement renvoyé par** `map`, pas sur un composant enfant.

```
<Grid container spacing={2}>
  {recipes.map((recipe) => (
    <Grid key={recipe.id} size={{ xs: 12, sm: 6, md: 4 }}>
      <RecipeCard recipe={recipe} />
    </Grid>
  ))}
</Grid>
```

filter() puis map()

```
const QuickRecipes = () => {
  return (
    <Grid container spacing={2}>
      {recipes
        .filter((recipe) => recipe.prepTime + recipe.cookTime <= 20)
        .map((recipe) => (
          <Grid key={recipe.id} size={{ xs: 12, sm: 6, md: 4 }}>
            <RecipeCard recipe={recipe} />
          </Grid>
        ))}
    </Grid>
  );
};
```

- `filter` réduit le tableau, `map` transforme le résultat en JSX
- Aucune des deux méthodes ne modifie `recipes`
- Les données restent la **source de vérité**, l'affichage en est déduit

Liste vide

Une collection peut être vide : il faut prévoir l'affichage correspondant.

```
const RecipeList = () => {
  if (recipes.length === 0) {
    return <Typography>Aucune recette pour le moment.</Typography>;
  }

  return (
    <Grid container spacing={2}>
      {recipes.map((recipe) => (
        <Grid key={recipe.id} size={{ xs: 12, sm: 6, md: 4 }}>
          <RecipeCard recipe={recipe} />
        </Grid>
      ))}
    </Grid>
  );
};
```

Le **retour anticipé** (guard clause) évite une condition autour de tout le JSX.

Récapitulatif Séance 10

- **Organisation** – `models/` pour les types, `data/` pour les données, `components/` pour l'affichage
- **Tableau de JSX** – React peut afficher un tableau de JSX
- **map()** – Transformer un tableau de données en tableau de JSX
- **key** – Identifiant unique et stable obligatoire dans les tableaux de JSX
- **filter** – Sélectionner des données avant de les afficher

Prochaine séance : Séance 11 – useState

Exercice filé S10

1. Créez `src/models/recipe.ts` (interfaces `Ingredient` et `Recipe`) et `src/models/category.ts` (interface `Category` : `id`, `name`, `description`)
2. Téléchargez les fichiers `recipes.ts` et `categories.ts` de la séance 10 sur moodle et placez-les dans `src/data` (ce sont les recettes du backend)
3. Modifiez `RecipeCard` pour recevoir une prop `recipe: Recipe` ; affichez le nom de la catégorie dans le `Chip` et les tags dans des `Chip` supplémentaires
4. Modifiez `RecipeList` pour afficher toutes les recettes du module de données avec `map` et `Grid`
5. Créez un composant `RecipeDetail` qui reçoit une recette et affiche toutes ses informations : description, temps, portions, ingrédients (`ul > li` : « 250 g farine ») et étapes (`ol > li`)
6. Affichez provisoirement `RecipeDetail` avec la première recette sous la liste (elle aura sa propre page en séance 15)
7. Vérifiez qu'aucun warning de `key` n'apparaît dans la console